

GeoScore

Studiedokument for jobbintervjuer · Skrevet på norsk · IN2000 Universitetet i Oslo, Team 20 · Karakter: A
Philip Fleischer, David Hovde, Julius Abdo, Matilda Wold Dahl, Veronica Corsepius Melen, Victoria Reinseth-Tollefsen

Innholdsfortegnelse

1. Prosjektoversikt og hva appen gjør **KJERNEPITCH**
2. Arkitektur — Clean Architecture og MVVM **KJERNEPITCH**
3. GeoScore-algoritmen **KJERNEPITCH**
4. Dependency Injection med Hilt **KJERNEPITCH**
5. Datalaget — DataSources, DTOs og Repositories **KJERNEPITCH**
6. Caching-strategi **KJERNEPITCH**
7. UI-laget — Jetpack Compose og UDF **KJERNEPITCH**
8. Brukerdesign og UX
9. AI-integrasjon med ChatGPT
10. Eksternale API-integrasjoner
11. Navigasjon og navigasjonsmønster
12. Agil metodikk og prosjektprosess
13. Teknologier og biblioteker
14. Intervjuspørsmål og svar

1. Prosjektoversikt og hva appen gjør

KJERNEPITCH

Hva er GeoScore?

GeoScore er en Android-applikasjon vi bygde i løpet av seks måneder i emnet IN2000 Programvareutvikling med prosjektarbeid ved Universitetet i Oslo. Teamet besto av seks studenter. Prosjektet ble vurdert til **karakter A**.

Appen hjelper folk som vurderer å kjøpe bolig å forstå klimarisikoen ved en adresse. Slagordet er: *"Vit hva du kjøper — før du kjøper det."*

Kjerneproblemet appen løser

Når du kjøper bolig i Norge i dag, er det vanskelig å finne ett samlet sted som gir deg svar på spørsmål som:

- Ligger huset i et flomrisikoområde?
- Er stedet utsatt for skred?
- Hvor mye ekstremvær har det vært her de siste 30 årene?
- Hva bør jeg gjøre for å beskytte huset mitt?

GeoScore samler geologisk og meteorologisk data fra offentlige norske datakilder og gir brukeren én samlet karakter (A–F) og en AI-generert tekstrapport med konkrete råd.

Hva kan brukeren gjøre i appen?

1. **Søke opp en adresse** via Geonorges adresse-API med live forslag mens man skriver.
2. **Se adressen på kart** med Google Maps, og eventuelt trykke på kartet for å velge en lokasjon.
3. **Generere en GeoScore** — appen henter vind- og nedbørsdata fra Frost API og sjekker om stedet er i flom- eller skredsone via NVE ArcGIS. Deretter beregnes en samlet score A–F.
4. **Lese en AI-rapport** — ChatGPT (GPT-4o) genererer en forklarende tekst med risikovurdering og tiltak basert på scorene.
5. **Utforske historisk klimadata** — temperaturer, nedbør, vind, snø og solskinn plottet i grafer (1991–2020-normaler).
6. **Lagre steder** for å komme raskt tilbake til dem.
7. **Se kartlag** fra NVE for flom, jordskred og steinskred direkte på kartet.

PITCH-TIPS

Start alltid med *problemet*, ikke teknologien. "Vi laget en app som hjelper boligkjøpere å forstå klimarisikoen ved en adresse — og vi fikk A." Det er en god åpning.

? **Mulig intervjusspørsmål:** *Hva er GeoScore, og hvorfor laget dere det?*

2. Arkitektur — Clean Architecture og MVVM

KJERNEPITCH

Overordnet prinsipp

Appen følger **Androids anbefalte lagdelte arkitektur**, som er inspirert av Clean Architecture. Hvert lag kjenner kun til laget under seg — aldri det over. Dette sikrer at forretningslogikken er uavhengig av rammeverk, og at man enkelt kan bytte ut implementasjoner uten å røre resten av koden.

```

UI-lag (Jetpack Compose + ViewModel)
    ↓
Domain-lag (Use Cases + Domenemodeller)
    ↓
Data-lag (Repositories + DataSources + Room)
  
```

Pakkestruktur

```

app/src/main/java/no/uio/ifi/in2000/team20/team20app/
├─ data/
│   ├─ api/           - API-endepunksdefinisjoner (Ktor-routes)
│   ├─ datasource/    - Direkte kommunikasjon med eksterne API-er
│   ├─ dto/           - Data Transfer Objects (serialiserte API-responser)
│   ├─ local/
│   │   └─ AppDatabase.kt
│   │   └─ dao/       - 11 Room DAOs
│   │       └─ entity/ - 11 Room-entiteter
│   └─ repository/    - Implementasjoner av repository-grensesnitt
├─ domain/
│   ├─ model/         - Rene Kotlin data-klasser (GeoScore, Location, Report...)
│   └─ usecase/       - 5 use cases med ett ansvarsområde hver
├─ ui/
│   ├─ screens/       - En pakke per skjerm (screen + ViewModel)
│   ├─ sharedViewModels/ - AppViewModel, FrostViewModel, SavedViewModel
│   ├─ navigation/    - NavigationRoot, Route-sealed-klasser
│   └─ components/    - Gjenbrukbare Compose-komponenter
  
```

```

└─ theme/          - Farger, typografi, Material 3-tema
└─ di/             - 4 Hilt-DI-moduler

```

MVVM-mønsteret (Model–View–ViewModel)

MVVM er designmønsteret vi bruker i UI-laget. Det deler ansvar på tre klare nivåer:

Del	I GeoScore	Ansvar
View	Jetpack Compose-skjermer (<code>GeoScoreScreen</code> , <code>SearchScreen</code> ...)	Vise data, ta imot brukerininput, kalle ViewModel-funksjoner
ViewModel	<code>GeoScoreViewModel</code> , <code>SearchViewModel</code> ...	Holde og oppdatere UI-state via <code>StateFlow<UiState></code>
Model	Use cases + Repositories + DataSources	Hente, bearbeide og levere data

Unidirectional Data Flow (UDF) — Énveis dataflyt

Et kjernepunkt i Android-arkitektur er at data flyter *én vei*. Dette gjør det lett å forstå og debugge appens tilstand:

```

Bruker trykker "Generer rapport"
  → Skjerm kaller geoScoreViewModel.load(location)
    → ViewModel oppdaterer _uiState til isScoreLoading = true
      → Skjerm re-rendrer automatisk (viser spinner)
    → ViewModel kaller getGeoScore.calculateGeoScore(lat, lon)
      → Use case henter data → returnerer GeoScore
    → ViewModel oppdaterer _uiState til geoScore = result
      → Skjerm re-rendrer og viser resultatet

```

Lav kobling — hva betyr det i praksis?

I GeoScore betyr lav kobling at ingenting "vet mer enn det trenger å vite":

- Alle repositories er definert som **grensesnitt (interface)** i domenelaget. ViewModels og use cases avhenger bare av grensesnittet, aldri av implementasjonen.

- Eksempel: `GetHazardScore` injiserer `ScoreCacheRepository` -grensesnittet og vet ingenting om Room, DAOs eller SQL.
- Hilt injiserer den korrekte implementasjonen automatisk ved oppstart.

Høy kohesjon — hva betyr det i praksis?

Høy kohesjon betyr at hver klasse gjør én ting, og gjør den godt:

- `GetHazardScore` beregner kun hazard-scoren. Den lagrer den og returnerer den — ingenting annet.
- `ScoreCacheRepository` eier all cache-logikk for scores. Use cases trenger ikke å tenke på Room.
- `ChatGPTRepositoryImpl` eier rapport-caching — den henter fra Room om mulig, ellers fra API.

? Mulig intervjusspørsmål: Kan du forklare MVVM og Clean Architecture i prosjektet ditt?

? Mulig intervjusspørsmål: Hva mener du med lav kobling og høy kohesjon? Gi et konkret eksempel fra koden din.

3. GeoScore-algoritmen **KJERNEPITCH**

Hva er GeoScore?

GeoScore er en samlet risikokarakter fra 0–100 (konvertert til A–F) som kombinerer tre uavhengige risikokomponenter. Jo høyere score, jo høyere risiko.

$$\text{GeoScore} = \text{HazardScore} \times 0,20 + \text{ExposureScore} \times 0,40 + \text{VulnerabilityScore} \times 0,40$$

De tre komponentene

1. HazardScore (Farepoengsum) — vekt: 20%

Måler *intensiteten* av ekstremvær — hvor mye over terskelen er vinden og nedbøren i gjennomsnitt?

- Terskelverdi for ekstremnedbør: **40 mm/dag**
- Terskelverdi for ekstremvind: **40 m/s vindkast**
- For nedbør: beregnes gjennomsnittlig overskridelse over terskelen for alle ekstremdager
- Dette normaliseres mot en referansemaksimum: `REF_MAX_PRECIP_OVER_THRESHOLD = 15 mm`
og `REF_MAX_WIND_OVER_THRESHOLD = 7 m/s`

```
// Fra GetHazardScore.kt
val extremePrecipDays = precipitationResults.values.filter { it > EXTREME_PRECIPITATION_THRESHOLD }

val precipOverTheThreshold = if (extremePrecipDays.isEmpty()) 0.0
    else extremePrecipDays.map { it - EXTREME_PRECIPITATION_THRESHOLD }.average()

val precipitationScore = min(100.0, (precipOverTheThreshold / REF_MAX_PRECIP_OVER_THRESHOLD) * 100)
val windScore = min(100.0, (windOverTheThreshold / REF_MAX_WIND_OVER_THRESHOLD) * 100)

// Hazard = nedbør vektet 65%, vind vektet 35%
val score = PRECIPITATION_WEIGHT * precipitationScore + WIND_WEIGHT * windScore
```

2. ExposureScore (Eksponeringspoengsum) — vekt: 40%

Måler *hyppigheten* av ekstremvær — hvor mange dager har det vært ekstremvær det siste halvørhundret?

- Teller alle unike dager med enten ekstremnedbør ($\geq 40\text{mm}$) ELLER ekstremvind ($\geq 40\text{ m/s}$)
- Normaliseres mot et spenn: $\text{MIN} = 0$, $\text{MAX} = 15 \text{ dager} \times 30 \text{ år} = 450$

```
// Fra GetExposureScore.kt
val extremePrecipDates = precipitationResults.filter { it.value >= EXTREME_PRECIPITATION_TH
val extremeWindDates   = windResults.filter { it.value >= EXTREME_WIND_GUST_THRESHOLD }.key

// toSet() fjerner duplikater (dager med BÅDE ekstremnedbør OG -vind telles én gang)
val sumOfextremeWeatherDates = (extremePrecipDates + extremeWindDates).toSet().size

val exposure = ((sumOfextremeWeatherDates - MIN_EXTREME_WEATHER_COUNT).toDouble()
                / (MAX_EXTREME_WEATHER_COUNT - MIN_EXTREME_WEATHER_COUNT)) * 100

val score = max(0.0, min(100.0, exposure))
```

3. VulnerabilityScore (Sårbarhetsscore) — vekt: 40%

Måler *geografisk risiko* — er stedet i en registrert flom- eller skredsone?

- Henter data fra NVE ArcGIS: `isInFloodZone` og `isInLandslideZone`
- I flomzone $\rightarrow$ floodScore = 100, ellers 0
- I skredsone $\rightarrow$ landslideScore = 100, ellers 0
- Vulnerability = landslide vekter 55%, flom vekter 45%

```
// Fra GetVulnerabilityScore.kt — parallelle API-kall med coroutines
val (isInFloodZone, isInLandslideZone) = coroutineScope {
    val floodDeferred      = async { zonesRepository.isInFloodZone(lat, lon) }
    val landslideDeferred  = async { zonesRepository.isInLandslideZone(lat, lon) }
    Pair(floodDeferred.await(), landslideDeferred.await())
}

val floodScore      = if (isInFloodZone)      100.0 else 0.0
val landslideScore  = if (isInLandslideZone) 100.0 else 0.0
val score = FLOOD_WEIGHT * floodScore + LANDSLIDE_WEIGHT * landslideScore
```


Karakterskalaen

A 0–16 Svært lav	B 17–32 Lav	C 33–49 Moderat	D 50–66 Høy	E 67–82 Svært høy	F 83–100 Kritisk
---	--	--	--	--	---

```
// Fra GeoScore.kt
fun scoreToGrade(score: Double?): String = when {
    score!! < 17 -> "A"
    score < 33   -> "B"
    score < 50   -> "C"
    score < 67   -> "D"
    score < 83   -> "E"
    score >= 83  -> "F"
    else        -> "?"
}
```

GetGeoScore — orkestrering med parallelle coroutines

`GetGeoScore` er use casen som setter det hele sammen. Den er et godt eksempel på effektiv bruk av Kotlin Coroutines:

```
// Fra GetGeoScore.kt
suspend fun calculateGeoScore(lat: Double, lon: Double): GeoScore {
    val locationKey = "%.2f, %.2f".format(lat, lon)

    // Sjekk cache først — unngå unødvendige API-kall
    val cachedTotal = scoreCacheRepository.getGeoScoreCache(locationKey)
    if (cachedTotal != null) return cachedTotal

    val observations: WindAndPrecipitationObservationsResult
    val vulnerabilityResult: VulnerabilityScoreResult

    coroutineScope {
        // observations og vulnerability kjøres PARALLELT
        val observationsDeferred = async { frostRepository.getWindAndPrecipitationObservations(lat, lon) }
        val vulnerabilityDeferred = async { getVulnerabilityScore.calculateVulnerabilityScore(lat, lon) }

        observations = observationsDeferred.await()
        vulnerabilityResult = vulnerabilityDeferred.await()
    }
}
```

```
// hazard og exposure avhenger av observations – kjøres etterpå
hazardResult = getHazardScore.calculateHazardScore(lat, lon, observations)
exposureResult = getExposureScore.calculateExposureScore(lat, lon, observations)
}

val geoScore = GeoScore(
    ...
    geoScore = (hazardResult.hazardScore * HAZARDScore_WEIGHT)
                + (exposureResult.exposureScore * EXPOSUREScore_WEIGHT)
                + (vulnerabilityResult.vulnerabilityScore * VULNERABILITYScore_WEIGHT)
)
scoreCacheRepository.saveGeoScore(geoScore)
return geoScore
}
```

NØKKELPOENG FOR INTERVJUET

Forklar at **observations og vulnerability kjøres parallelt** fordi de ikke avhenger av hverandre. Hazard og exposure kan ikke starte før observations er ferdig. Dette er en bevisst arkitekturelt valg for ytelse.

? Mulig intervju spørsmål: *Hvordan fungerer GeoScore-algoritmen din?*

? Mulig intervju spørsmål: *Hvorfor har dere valgt disse vektene (20/40/40)?*

4. Dependency Injection med Hilt

KJERNEPITCH

Hva er Dependency Injection?

Dependency Injection (DI) er et mønster der objekter ikke lager sine egne avhengigheter, men får dem levert utenfra. I stedet for at `GeoScoreViewModel` lager en instans av `GetGeoScore` selv, får den den injisert via konstruktøren.

Fordeler:

- Lett å teste — man kan injisere falske implementasjoner (mocks)
- Sentralisert konfigurasjons — en plass bestemmer hvilken implementasjon som brukes
- Ingen direkte kobling mellom klasser

Hvordan Hilt er satt opp i GeoScore

Hilt er Googles anbefalte DI-rammeverk for Android, bygget på Dagger. Vi bruker versjon 2.59.2.

Annotasjon	Brukt på	Hva den gjør
<code>@HiltAndroidApp</code>	<code>GeoScoreApp</code>	Initialiserer Hilt for hele appen
<code>@AndroidEntryPoint</code>	<code>MainActivity</code>	Gjør det mulig å injisere i Activity
<code>@HiltViewModel</code>	Alle ViewModels	ViewModels kan bruke constructor injection
<code>@Inject</code> <code>constructor</code>	Repos, datasources, use cases	Hilt håndterer instansiering
<code>@Singleton</code>	Nettverksklienter, DAOs, repositories	Én instans deles gjennom hele appen

De fire Hilt-modulene

NetworkModule — nettverksklienter

```
// Fra NetworkModule.kt
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    @FrostClient          // Qualifier — skiller mellom de tre klientene
    @Provides
    @Singleton
    fun provideFrostHttpClient(): HttpClient = HttpClient(CIO) {
        install(ContentNegotiation) {
            json(Json { ignoreUnknownKeys = true })
        }
        expectSuccess = false    // Kaster ikke exception på 4xx/5xx
        defaultRequest {
            header("User-Agent", "IN2000-Team20 jeryosa@uio.no")
        }
    }

    @Provides
    @Singleton
    fun provideOpenAIClient(): OpenAI = OpenAI(Constants.CHATGPT_API_KEY)
}
```

Vi bruker `@Qualifier` -annotasjoner (`@FrostClient` , `@GeoSearchClient` , `@NveZonesClient`) for å skille mellom de tre ulike Ktor HttpClient-instansene, siden alle er av samme type.

DatabaseModule — Room-databasen

```
// Konseptuelt — gir ut Room-databasen og alle 11 DAOs som singletons
@Provides @Singleton fun provideDatabase(app: Application): AppDatabase =
    Room.databaseBuilder(app, AppDatabase::class.java, "team20_app_db").build()

@Provides @Singleton fun provideTemperatureCacheDao(db: AppDatabase) = db.temperatureCacheDao
```

DataSourceModule og RepositoryModule

Disse bruker `@Binds` for å fortelle Hilt at når noen ber om et grensesnitt, skal den korrekte implementasjonen leveres:

```
// RepositoryModule.kt — binder grensesnitt til implementasjon
@Binds abstract fun bindFrostRepository(impl: FrostRepository): FrostRepositoryService
@Binds abstract fun bindNveZonesRepository(impl: NveZonesRepository): NveZonesRepositoryService
@Binds abstract fun bindScoreCacheRepository(impl: ScoreCacheRepositoryImpl): ScoreCacheRepository
```

? **Mulig intervju spørsmål:** *Hva er Dependency Injection, og hvorfor er det nyttig?*

? **Mulig intervju spørsmål:** *Hva er forskjellen på @Provides og @Binds i Hilt?*

5. Datalaget — DataSources, DTOs og Repositories

KJERNEPITCH

De tre lagene i datalaget

DataSource — rå API-kall

DataSources er klassene som snakker direkte med eksterne API-er. De vet ingenting om domenemodeller — de returnerer rå DTOer. Eksempel fra `FrostDataSource`:

```
// Fra FrostDataSource.kt — henter rådata fra Frost V0
override suspend fun getTemperatureNormals(lat: Double, lon: Double, sources: String): FrostData {
    return client.get(FrostRoutes.OBSERVATIONS_V0) {
        url.encodedParameters.append("sources", sources)
        url.encodedParameters.append("elements",
            "mean(air_temperature P1M),mean(max(air_temperature P1D) P1M),mean(min(air_temperature P1D) P1M)"
        )
        url.encodedParameters.append("referencetime", "1991-01-01/2020-12-31")
        header("Authorization", authHeader)
    }.frostBody() // Kaster exception med Frost's feilmelding ved ikke-2xx
}
```

For **NVE ArcGIS** sender vi koordinater som geometri-parameter til et ArcGIS-tjenestekall:

```
// Fra NveZonesRemoteDataSource.kt
override suspend fun getFloodZoneData(lon: Double, lat: Double): ArcGisResponseDto {
    return client.get(NveRoutes.FLOM) {
        parameter("geometry", """"{"x":$lon,"y":$lat,"spatialReference":{"wkid":4326}}""")
        parameter("geometryType", "esriGeometryPoint")
        parameter("spatialRel", "esriSpatialRelIntersects")
        parameter("returnGeometry", false)
        parameter("f", "json")
    }.body()
}
```

DTOer (Data Transfer Objects)

DTOer er Kotlin data-klasser som speiler JSON-strukturen fra API-ene. De er annotert med

`@Serializable` (kotlinx.serialization) slik at Ktor kan deserialisere dem automatisk:

```
// Eksempel på DTO-struktur
@Serializable
data class FrostV0ObservationResponseDto(
    val data: List<ObservationEntry>
)

@Serializable
data class ObservationEntry(
    val referenceTime: String,          // "1991-01-01T00:00:00.000Z"
    val observations: List<Observation>
)

@Serializable
data class Observation(
    val elementId: String,             // "mean(air_temperature PlM)"
    val value: Double
)
```

Repositories — kartlegger data til domenemodeller

Repositories er mellomlaget mellom datalaget og domenelaget. De:

1. Kaller DataSource for rådata
2. Aggregerer og bearbeider rå-DTOen til noe meningsfylt
3. Cacher resultatet i Room (eller in-memory)
4. Returnerer domenemodeller

Et klassisk eksempel er `FrostRepository.aggregateByMonthV0()` — en funksjon som tar 360 rådata-observasjoner (30 år × 12 måneder) og aggregerer dem til 12 månedlige normaler:

```
// Fra FrostRepository.kt — extension function på DTO
private fun FrostV0ObservationResponseDto.aggregateByMonthV0(elementId: String): Map<Int, Double> {
    return data
        .mapNotNull { entry ->
            // Trekk ut månedsnummer fra ISO 8601-tidsstempel (tegn 5-6)
            val month = entry.referenceTime.substring(5, 7).toIntOrNull() ?: return@mapNotNull null
            val value = entry.observations.find { it.elementId == elementId }?.value
            month to value
        }
```

```

    }
    .groupBy({ it.first }, { it.second })
    .mapValues { (_, values) -> values.average() } // Gjennomsnitt per måned over 30 år
}

```

VIKTIG SKILLE

DTOer er kun i datalaget og bærer rå JSON-data. **Domenemodeller** (som `GeoScore` , `Location`) er i domenelaget og er fri for alle Android- og nettverksavhengigheter.

Repositories oversetter mellom disse to verdenene.

Room — lokal database

Room er Androids ORM (Object-Relational Mapping) over SQLite. Vi bruker det for å cache data lokalt.

Databasen heter `team20_app_db` og har 11 tabeller:

Tabell	Innhold	Eier
<code>saved_locations</code>	Brukerens lagrede adresser	SavedRepository
<code>hazard_cache</code>	Beregnet hazard-score per lokasjon	ScoreCacheRepository
<code>exposure_cache</code>	Beregnet eksponerings-score	ScoreCacheRepository
<code>vulnerability_cache</code>	Beregnet sårbarhetsscore	ScoreCacheRepository
<code>total_score_cache</code>	Samlet GeoScore	ScoreCacheRepository
<code>report_cache</code>	AI-generert rapport fra ChatGPT	ChatGPTRepository
<code>temperature_cache</code>	Temperatur-normaler fra Frost	FrostRepository
<code>wind_cache</code>	Vinddata fra Frost	FrostRepository
<code>sunshine_cache</code>	Solskinndata fra Frost	FrostRepository
<code>snow_cache</code>	Snødybde data fra Frost	FrostRepository

Hvert sett (entity + DAO) følger samme mønster:

```
// Entity — definerer en rad i databasen
@Entity(tableName = "temperature_cache")
data class TemperatureCacheEntity(
    @PrimaryKey val stationId: String,
    val monthlyMean: String,    // JSON-serialisert List<Double>
    val monthlyMax: String,
    val monthlyMin: String
)

// DAO — spørringsgrensesnitt
@Dao
interface TemperatureCacheDao {
    @Query("SELECT * FROM temperature_cache WHERE stationId = :key")
    suspend fun getByKey(key: String): TemperatureCacheEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(entity: TemperatureCacheEntity)
}
```

? Mulig intervju spørsmål: *Hva er en DTO, og hvorfor trenger vi dem?*

? Mulig intervju spørsmål: *Hva er Room, og hva bruker dere det til?*

? Mulig intervju spørsmål: *Hva er forskjellen på en Entity og en DAO i Room?*

6. Caching-strategi

KJERNEPITCH

Hvorfor cache?

GeoScore-beregningen gjør mange API-kall: Frost V1 (vind + nedbør over 30 år), NVE ArcGIS (to kall), og ChatGPT. Uten caching ville en allerede besøkt adresse ta like lang tid å laste andre gang. Med caching returneres resultatet umiddelbart.

I tillegg er historiske normaler (1991–2020) *uforanderlige* — de trenger aldri å utløpe, noe som gjør caching særlig naturlig her.

To-lags caching i FrostRepository

Lag 1: In-memory cache (stasjons-ID)

Koordinater rundes til 2 desimaler (~1 km presisjon) og brukes som nøkkel i en `mutableMapOf` i minnet. Når vi kaller `getTemperatureData`, `getWindData` og de andre metodene for samme lokasjon, gjøres kun ett API-kall for å finne nærmeste stasjon:

```
// Fra FrostRepository.kt
private val stationCache = mutableMapOf<String, String>()

private suspend fun getOrCacheStations(lat: Double, lon: Double): String {
    val key = "%.2f, %.2f".format(lat, lon)
    return stationCache[key] ?: run {
        val stations = dataSource.getStationsNearby(lat, lon)
        stationCache[key] = stations
        stations
    }
}
```

Lag 2: Room-cache (klimadata, indeksert på stasjon-ID)

Klimadataene caches i Room med nærmeste stasjon-ID som primærnøkkel. To lokasjoner som deler nærmeste stasjon gjenbruker cachet data uten et nytt API-kall:

```
// Fra FrostRepository.getTemperatureData()
val stations = getOrCreateCacheStations(lat, lon)
val stationId = stations.split(",").first() // Nærmeste stasjon som cache-nøkkel

val cached = temperatureCacheDao.getByKey(stationId)
if (cached != null) {
    // Cache-treff: returner umiddelbart uten nettverkskall
    return@runCatching Triple(fromJson(cached.monthlyMean), ...)
}

// Cache-miss: hent fra API, lagre i Room
val data = dataSource.getTemperatureNormals(lat, lon, stations)
temperatureCacheDao.insert(TemperatureCacheEntity(stationId = stationId, ...))
Triple(meanList, maxList, minList)
```

Solskinn som spesialtilfelle

Kun 36 av Norges hundrevis av meteorologiske stasjoner måler solskinn. Derfor gjøres et eget stasjonssøk bare for solskinn, og resultatet caches separat i `sunshineStationCache` (in-memory) og `sunshineCacheDao` (Room):

```
// Trinn 1: hent alle stasjons-IDer som har solskinndata (1991-2020)
val availableResponse: FrostV0AvailableTimeSeriesResponseDto =
    client.get(FrostRoutes.AVAILABLE_TIMESERIES_V0) {
        append("elements", "sum(duration_of_sunshine PlM)")
        ...
    }

// Trinn 2: finn nærmeste av disse 36 stasjonene
val sourcesResponse = client.get(FrostRoutes.SOURCES_V0) {
    append("geometry", "nearest(POINT($lon $lat))")
    append("ids", allSunshineIds)
    append("nearestmaxcount", "1")
    ...
}
```

Score-caching med ScoreCacheRepository

Alle tre use cases sjekker Room-cachen med `locationKey = "%.2f, %.2f".format(lat, lon)` som nøkkel. Samme nøkkelformat brukes konsekvent gjennom hele appen:

```
// Mønster som brukes i alle use cases
val locationKey = "%.2f, %.2f".format(lat, lon)
val cachedResult = scoreCacheRepository.getHazardCache(locationKey)
if (cachedResult != null) return cachedResult // Hurtig returnering fra cache
// ... beregning ...
scoreCacheRepository.saveHazardScore(locationKey, result)
return result
```

KJENT BEGRENSNING

`FrostRepository` aggregerer klimanormaler fra de 10 nærmeste stasjonene, men cacher med kun den *nærmeste* stasjons-ID som nøkkel. Teknisk sett representerer den cachede dataen et gjennomsnitt av 10 stasjoner, ikke én. Dette er en akseptert avveining innenfor tidsrammen, men bør adresseres ved videreutvikling.

? **Mulig intervjusspørsmål:** *Hvordan håndterer dere caching i appen?*

? **Mulig intervjusspørsmål:** *Hva er fordelene og ulempene med din caching-strategi?*

7. UI-laget — Jetpack Compose og UDF

KJERNEPITCH

Jetpack Compose

Jetpack Compose er Googles moderne, deklorative UI-rammeverk for Android. I stedet for å manipulere XML-layouts og Views imperative (endre én ting av gangen), beskriver man i Compose *hvordan UIet skal se ut* som en funksjon av tilstanden. Når tilstanden endres, re-rendres UI automatisk.

```
// Composables er funksjoner annotert med @Composable
@Composable
fun GeomarkingBadge(grade: String, ...) {
    val badgeColor = when (grade.uppercase()) {
        "A" -> Color(0xFF4CAF50)    // Grønn
        "B" -> Color(0xFF8BC34A)
        "C" -> Color(0xFFFFC107)    // Gul
        "D" -> Color(0xFFFFC56B)
        "E" -> Color(0xFFEFA066)
        "F" -> Color(0xFFE36C5C)    // Rød
        else -> Color(0xFFFFC107)
    }
    Card(shape = CircleShape, colors = CardDefaults.cardColors(containerColor = badgeColor)) {
        Text(text = grade, fontWeight = FontWeight.Bold, color = Color.White)
    }
}
```

StateFlow og collectAsStateWithLifecycle

ViewModel holder UI-state i en `MutableStateFlow`, som eksponeres som en `StateFlow` til skjermen. Composables observerer denne via `collectAsStateWithLifecycle()`:

```
// ViewModel — holder state privat og mutable
private val _uiState = MutableStateFlow(GeoScoreUiState())
val uiState: StateFlow<GeoScoreUiState> = _uiState.asStateFlow()

// Composable — observer med livssyklus-bevissthet
```

```

val geoState by geoScoreViewModel.uiState.collectAsStateWithLifecycle()

// UI re-rendres automatisk når geoState endres
if (geoState.isScoreLoading) {
    LoadingState(message = listOf("Henter data...", "Nesten ferdig..."))
    return@Scaffold
}
if (geoState.scoreError != null) {
    ErrorState(message = "Ingen internettforbindelse.", onRetry = { geoScoreViewModel.load
    return@Scaffold
}
// Vis resultater
GeomarkingCard(location = location, geoState = geoState, ...)

```

`collectAsStateWithLifecycle()` er viktig: den stopper innsamling av data når skjermen ikke er synlig (bakgrunn/minimert), noe som sparer batteri og unngår minnelekkasjer.

LaunchedEffect — sideeffekter i Compose

Fordi Composables er "rene" funksjoner (de bare beskriver UI), brukes `LaunchedEffect` til å starte asynkrone operasjoner som ikke er direkte del av UI-beskrivelsen:

```

// Fra GeoScoreScreen.kt – laster data når skjermen vises med en gitt lokasjon
LaunchedEffect(location) {
    frostViewModel.loadFrostStats(location)
    savedViewModel.checkIfSaved(location)
    geoScoreViewModel.load(location)
}

```

`LaunchedEffect(location)` betyr: "Kjør dette koroutine-blokket én gang, og kjør det på nytt hvis `location` endres."

UiState som sealed class / data class

Hver skjerm har sin egen `UiState` -data-klasse som representerer alle mulige tilstander:

```

data class GeoScoreUiState(
    val isScoreLoading: Boolean = false,    // Viser spinner
    val geoScore: GeoScore? = null,        // Null = ikke lastet ennå
    val grade: String = "",

```

```

val scoreError: String? = null,          // Ikke-null = noe gikk galt
val isReportLoading: Boolean = false,
val aiReport: Report? = null,
val reportError: String? = null
)

```

Med `.update { it.copy(...) }` oppdateres kun de relevante feltene:

```

// ViewModel oppdaterer state atomisk
_uiState.update { it.copy(isScoreLoading = true, scoreError = null) }

```

viewModelScope og Dispatchers

All nettverkskommunikasjon og IO skjer utenfor main thread via Kotlin Coroutines:

```

// Fra GeoScoreViewModel.kt
fun load(location: Location) {
    viewModelScope.launch(Dispatchers.IO) { // Flytt til IO-tråd
        _uiState.update { it.copy(isScoreLoading = true) }
        runCatching { getGeoScore.calculateGeoScore(location.lat, location.lon) }
            .fold(
                onSuccess = { score -> _uiState.update { it.copy(geoScore = score, ...) } },
                onFailure = { e -> _uiState.update { it.copy(scoreError = e.message) } }
            )
    }
}

```

? Mulig intervju spørsmål: *Hva er Jetpack Compose, og hvordan skiller det seg fra tradisjonell Android-utvikling?*

? Mulig intervju spørsmål: *Hva er StateFlow, og hvorfor bruker man det i Android?*

8. Brukerdesign og UX

Designfilosofi

Vi fulgte **Material 3** (Material You) designsystemet — Googles moderne retningslinjer for Android-design. Det gir konsistent typografi, farger (inkludert dynamisk temafarging), komponenter og bevegelse.

Adaptivt layout — responsiv for ulike skjermstørrelser

En av de mer avanserte UX-løsningene er `AdaptiveNavigationScaffold`, som automatisk bytter mellom:

- **NavigationBar** (bunnlinje) på kompakte skjermer (telefoner)
- **NavigationRail** (sidesøyle) på medium/store skjermer (nettbrett, foldable)

```
// Fra AdaptiveNavigationScaffold.kt
val compactScreenWidth = !LocalWindowSizeClass.current
    .isWidthAtLeastBreakpoint(WindowSizeClass.WIDTH_DP_MEDIUM_LOWER_BOUND)

val layoutType = if (!compactScreenWidth) NavigationSuiteType.NavigationRail
    else NavigationSuiteType.NavigationBar
```

Hjemskjermen er også adaptiv — på brede skjermer vises innhold i 2 kolonner via

`LazyVerticalGrid`:

```
LazyVerticalGrid(
    columns = GridCells.Fixed(if (compactScreenWidth) 1 else 2)
)
```

Tilgjengelighet (Accessibility)

Vi har implementert semantikk-informasjon for skjermlesere (TalkBack) på kritiske steder:


```
// Tydelig handlingsbeskrivel for bookmark-knappen
IconButton(
    modifier = Modifier.semantics {
        onClick(
            label = if (isCurrentSaved) "Remove address from saved" else "Save address",
            action = { onSaveToggle(isCurrentSaved); true }
        )
    }
)

// Heading-markering for skjermleser-navigasjon
Text("Vit hva du kjøper", modifier = Modifier.semantics { heading() })

// Grupper semantisk relaterte elementer
Column(modifier = Modifier.semantics(mergeDescendants = true) {})
```

Navigasjonsflyt — brukerreisen

Vi har tenkt grundig på brukerreisen. Typisk flyt:

1. **Hjemskjerm** → 2. **Søkeskjerm** (tastatur åpnes automatisk) → 3. **Kartskjerm** (adresse markert med pin) → 4. **GeoScore-skjerm** (rapport) → 5. **Klimadata-skjerm** (grafer)

Søkefunksjonalitet med debounce

Søkefeltet bruker **debouncing** for å ikke sende et API-kall for hvert tastetrykk — det ventes 300ms etter siste tastefeil:

```
// Fra SearchViewModel.kt
companion object {
    private const val DEBOUNCE_DELAY_MS = 300L
}

fun updateInput(text: String) {
    searchJob?.cancel() // Avbryt forrige søk
    searchJob = viewModelScope.launch {
        delay(DEBOUNCE_DELAY_MS) // Vent 300ms
        performSearch(text) // Søk bare om ingenting nytt skrives
    }
}
```

UX for lasting og feil

Vi har gjenbrukbare komponenter for alle mellomtilstander:

- `LoadingState` — viser en animert spinner med vekselnde tekstmeldinger ("Henter data...", "Nesten ferdig...")
- `ErrorState` — viser feilmelding med "Prøv igjen"-knapp
- `EmptyState` — vises når en liste er tom
- `ExpandableInfoBox` — for kollapsbare seksjoner i rapporten

? Mulig intervju spørsmål: *Hvordan har dere tenkt på brukeropplevelse i GeoScore?*

? Mulig intervju spørsmål: *Hva er adaptivt layout, og hvordan er det implementert?*

9. AI-integrasjon med ChatGPT

Arkitekturen for AI-rapporten

Etter at GeoScore er beregnet, sendes scorene til ChatGPT (GPT-4o) for å generere en forklarende rapport i fire deler: nedbør, vind, flom og skred.

Dataflyten:

```
GeoScoreViewModel.loadReport(score)
  → GetAiReport.generateReport(geoScore)
    → ChatGPTRepository.generateReport(geoScore)
      └─ Sjekk Room-cache (reportCacheDao)
        └─ Cache-treff: returner umiddelbart
      └─ Cache-miss: ChatGPTRemoteDataSource.getReport(geoScore)
        → OpenAI SDK → GPT-4o → parse svar → lagre i Room → returner
```

Prompt engineering

Vi har designet en nøye strukturert prompt som:

- Definerer rollen ("Du er en klimarisiko-ekspert")
- Beskriver karakterskalaen eksplisitt (0–16 = A, 17–32 = B...)
- Injiserer de fire scorene dynamisk
- Spesifiserer eksakt format — fire avsnitt separert med `----`
- Instruerer om håndtering av `INGEN_DATA`-tilfeller
- Setter ordgrenser og tone

```
// Fra ChatGPTRemoteDataSource.kt — forkortet
content = "Du er en klimarisiko-ekspert...\n" +
  "Scorer for denne lokasjonen:\n" +
  "- Nedbør: ${geoScore.precipitationScore ?: "INGEN_DATA"}\n" +
  "- Vind/storm: ${geoScore.windScore ?: "INGEN_DATA"}\n" +
  "- Flom: ${geoScore.floodScore}\n" +
  "- Skred: ${geoScore.landslideScore}\n" +
```

```
"\nSkriv FIRE separate avsnitt separert med ---\n" +
"Hvert avsnitt skal:\n" +
"- Være 2-3 setninger\n" +
"- Inneholde punktliste med tiltak hvis karakteren er lavere enn B"
```

Parsing av GPT-svaret

ChatGPT returnerer én lang streng. Vi parser den ved å splitte på separatoren `---`:

```
// Fra ChatGPTRepositoryImpl.kt
val result = dataSource.getReport(geoScore)
val parts = result?.split("---")

return Report(
    locationKey = geoScore.locationKey,
    extremePrecipitationText = parts?.getOrNull(0)?.trim() ?: "",
    extremeWindText          = parts?.getOrNull(1)?.trim() ?: "",
    floodText                 = parts?.getOrNull(2)?.trim() ?: "",
    landslideText             = parts?.getOrNull(3)?.trim() ?: ""
)
```

AI-rapport caching

ChatGPT-kall er dyre (tid og penger). Rapporten caches permanent i Room under `report_cache`-tabellen. Andre gang man ser på samme adresse kommer rapporten umiddelbart:

```
override suspend fun generateReport(geoScore: GeoScore): Report {
    val cachedReport = reportCacheDao.getByKey(geoScore.locationKey)
    if (cachedReport != null) return cachedReport.toDomain() // Umiddelbar retur fra cache

    val report = getAiGeneratedReport(geoScore) // API-kall bare ved cache-miss
    reportCacheDao.insert(report.toEntity())
    return report
}
```

SNAKKEPUNKT

GPT-4o brukes fremfor GPT-3.5 fordi det gir mer presise og norske svar. AI-integrasjonen er et godt eksempel på at appen ikke bare henter data, men tolker og kommuniserer data på en menneske-lesbar måte.

10. Eksternale API-integrasjoner

API	Hva det gir	Autentisering
Frost Vo (met.no)	Historiske klimanormaler 1991–2020: temperatur, vind, nedbør, snø, solskinn fra nærmeste målestasjon	Basic Auth (client ID + secret, base64-encoded)
Frost V1 RC (met.no)	Daglige observasjoner 1990–2020 for GeoScore- beregning (nedbør og vindkast per dag)	Basic Auth (samme nøkler)
NVE ArcGIS	Sjekker om koordinat ligger innenfor kartlagt flom- eller skredzone	Ingen — offentlig tjeneste
George adresse-API	Live adresseforslag på norske adresser → returnerer koordinater	Ingen — offentlig tjeneste
OpenAI (GPT-4o)	Genererer norsk forklaringstekst og tiltak basert på scorene	API-nøkkel (Bearer token)
Google Maps	Interaktivt kart med pins, WMS-kartlag (NVE- soner visuelt)	Google Maps API-nøkkel

Frost V1 — ranked observations

Frost V1 er en nyere API-versjon som returnerer de *best rangerte* observasjonene nær et koordinat. Vi prøver tre radier parallelt (10, 20, 30 km) og bruker den nærmeste som returnerer data:

```
// Fra FrostDataSource.kt - parallelle fallback-søk
override suspend fun getRankedObservationsForPrecipitation(...): FrostV1ResponseDto = corou
    val d10 = async { fetchV1(lat, lon, 10.0, ...) } // 10 km radius
    val d20 = async { fetchV1(lat, lon, 20.0, ...) } // 20 km radius
    val d30 = async { fetchV1(lat, lon, 30.0, ...) } // 30 km radius

    val (r10, r20, r30) = awaitAll(d10, d20, d30) // Alle tre kjøres parallelt

    when {
        r10.data.tseries.isNotEmpty() -> r10 // Foretrekk nærmeste
```

```
r20.data.tseries.isEmpty() -> r20
r30.data.tseries.isEmpty() -> r30
else -> FrostV1ResponseDto()           // Tom respons – ingen data
}
}
```

For vind prøves først vindkast (`max(wind_speed_of_gust P1D)`). Hvis ingen stasjon har dette, faller vi tilbake til gjennomsnittlig vindhastighet (`mean(wind_speed P1D)`).

Ktor som HTTP-klient

Vi bruker Ktor (3.4.3) med CIO-engine for alle HTTP-kall. Ktor er Kotlin-first, koroutine-basert og main-safe. Serialisering gjøres med `kotlinx.serialization` .

11. Navigasjon og navigasjonsmønster

Navigation 3 — type-safe navigasjon

Vi bruker **Navigation 3** (versjon 1.1.1) — Jetpack Navigation for Compose. Rutene er definert som **sealed-klasser** som arver fra `Route`, noe som gir type-safe navigasjon og forhindrer runtime-feil fra feil strenger:

```
// Route.kt
sealed interface Route : NavKey {
    data object HomeDestination      : Route
    data object MapDestination       : Route
    data object SavedDestination     : Route
    data object SearchDestination    : Route
    data class GeoscoreDestination(val location: Location)      : Route
    data class ClimateStatsDestination(val location: Location) : Route
}
```

At `GeoscoreDestination` har `location` som konstruktørparameter betyr at lokasjonen sendes direkte som objekt — ikke som streng i en URL-sti slik man måtte gjøre med Navigation 2.

Tilbake-navigasjon fra KlimaStatsScreen

Et interessant navigasjonsdetalj: Når brukeren trykker tilbake fra klimadataskjermen, hopper de over GeoScore-skjermen og returnerer til kart/lagrede steder:

```
// Fra NavigationRoot.kt
onBackClick = {
    val geoScoreIndex = backStack.indexOfLast { it is Route.GeoscoreDestination }
    if (geoScoreIndex > 0) {
        while (backStack.size > geoScoreIndex) {
            backStack.removeLastOrNull() // Fjern klimadata- og geoscore-destinasjonen
        }
    } else { goBack() }
}
```


Delte ViewModels

Tre ViewModels deles mellom skjermer og instansieres på Activity-nivå i `NavigationRoot` :

- `AppViewModel` — holder valgt lokasjon tilgjengelig overalt. Bruker `SavedStateHandle` for å overleve prosessdød (OS-drept bakgrunnsapp).
- `FrostViewModel` — `HomeScreen` starter forhåndslasting av klimadata mens brukeren ser på kartet, slik at grafene er klare når brukeren trykker "Historisk klimadata".
- `SavedViewModel` — `isCurrentSaved` -flagget må være konsistent overalt (bookmark-ikonet).

12. Agil metodikk og prosjektprosess

Scrum-tilnærming

Vi fulgte en tilpasset Scrum-metode gjennom seks måneder. Prosjektloggen (*Prosesslogg_IN2000_V26.pdf*) og sluttrapporten (*team20_rapport.pdf*) dokumenterer prosessen i detalj.

- **Sprinter** — prosjektet var delt inn i iterasjoner der vi planla, implementerte og evaluerte.
- **Standup-møter** — regelmessige kortmøter for å synkronisere fremdrift.
- **Retrospektiver** — etter hver sprint reflekterte vi over hva som fungerte og hva som ikke fungerte.
- **Issues og Pull Requests** på GitHub — alle endringer gikk gjennom code review.

Teamavtalen

Vi inngikk en formell teamavtale (*20_teamavtale.pdf*) ved prosjektstart som definerte forventninger, kommunikasjonsrutiner og konflikthåndtering.

Rollefordeling

Teamet på seks hadde ulike ansvarsområder som ble rotert og justert gjennom prosjektet. En viktig lærdom var å kommunisere arkitekturvalg tidlig for å unngå motstridende implementasjoner.

Tekniske utfordringer underveis

- **Frost API-versjonering** — Frost V0 og V1 returnerer ulike dataformater. Vi måtte bygge to separate integrasjoner og aggregeringslogikk for V0 client-side.
- **Solskinndata** — Bare 36 norske stasjoner har solskinndata. Vi oppdaget dette sent og måtte legge til et eget to-steps oppslag.
- **Hilt og AGP 9.0** — Hilt 2.57 er inkompatibel med Android Gradle Plugin 9.0 (bruker `BaseExtension` som ble fjernet). Vi måtte oppgradere til Hilt 2.59+.

- **Navigation 3** — Relativt ny versjon uten mye dokumentasjon. Vi lærte mye gjennom prøving og feiling.

? **Mulig intervjusspørsmål:** *Hvilke tekniske utfordringer møtte dere, og hvordan løste dere dem?*

? **Mulig intervjusspørsmål:** *Hva lærte du av å jobbe i et team på seks?*

? **Mulig intervjusspørsmål:** *Hva ville du gjort annerledes neste gang?*

13. Teknologier og biblioteker

Teknologi	Versjon	Hva den gjør
Kotlin	2.3.21	Kodespråk — null-safety, extension functions, coroutines, data classes
Jetpack Compose	BOM 2026.05.00	Deklarativt UI-rammeverk — erstatter XML-layouts
Material 3	—	Googles designsystem — farger, typografi, komponenter
Navigation 3	1.1.1	Type-safe navigasjon mellom Compose-skjermer
Hilt	2.59.2	Dependency Injection — bygget på Dagger
KSP	2.3.8	Kotlin Symbol Processing — code generation for Hilt og Room
Room	2.8.4	ORM over SQLite — lokal database og caching
Ktor	3.4.3	Kotlin-first HTTP-klient, CIO-engine, koroutine-basert
kotlinx.serialization	1.11.0	JSON-serialisering / deserialisering av API-responser
OpenAI client	4.1.0	Kotlin SDK for OpenAI API (ChatGPT)
Google Maps Compose	8.3.0	Interaktivt kart med Compose-API
Kotlin Coroutines	1.11.0	Asynkrone operasjoner uten callback-helvete
Coil	—	Bildehåndtering i Compose
JUnit 4	4.13.2	Unit testing

minSdk	24 (Android 7.0)	Støtter ~95% av aktive Android-enheter
targetSdk / compileSdk	36	Nyeste Android-funksjonalitet og sikkerhet

Kotlin Coroutines — kort forklaring

Coroutines er Kotlin's mekanisme for asynkron programmering. De er lette tråder som kan suspenderes og gjenopptas uten å blokkere en ekte OS-tråd.

- `suspend fun` — en funksjon som kan pause uten å blokkere tråden
- `coroutineScope { }` — lager en ny koroutine-kontekst der alle barn ventes på
- `async { }` — starter en koroutine som returnerer en `Deferred<T>`
- `await()` — venter på resultatet fra `async`
- `awaitAll()` — venter på flere `Deferred` parallelt
- `launch { }` — starter en koroutine der man ikke trenger returverdien

14. Intervjuspørsmål og anbefalte svar

Om prosjektet generelt

SPØRSMÅL: "FORTELL MEG OM GEOSCORE"

GeoScore er en Android-app jeg laget sammen med fem andre studenter i løpet av seks måneder i IN2000 ved UiO. Vi fikk A. Appen hjelper folk som vurderer å kjøpe bolig å forstå klimarisikoen ved en adresse — om det er flomfare, skredutsatt, og om det har vært ekstremvær de siste 30 årene. Vi kombinerer data fra Meteorologisk institutts Frost-API, NVEs kartdata og ChatGPT for å gi en samlet risikokarakter (A–F) og en forklarende tekstrapport.

SPØRSMÅL: "HVA VAR DIN ROLLE?"

Jeg var med på å designe og implementere [din faktiske bidrag her — f.eks. algoritmen, UI, caching, API-integrasjoner]. I tillegg var jeg involvert i arkitekturvalg for hele appen.

Om arkitektur og designmønstre

SPØRSMÅL: "FORKLAR MVVM"

MVVM er et arkitekturmønster som deler UI-koden i tre lag: View er Compose-skjermene som viser data. ViewModel holder UI-state som en StateFlow og tar imot hendelser fra View. Model er domenelaget — use cases og repositories som henter og bearbeider data. Data flyter alltid nedover: view → ViewModel → domain/data, og tilstand flyter alltid tilbake som en StateFlow. View re-rendres automatisk når tilstanden endres.

SPØRSMÅL: "HVA ER DEPENDENCY INJECTION?"

DI er et mønster der en klasse ikke lager sine egne avhengigheter selv, men får dem levert utenfra. I GeoScore bruker vi Hilt — GeoScoreViewModel trenger GetGeoScore og GetAiReport. I stedet for at ViewModelen lager disse selv, ber den om dem via konstruktøren, og Hilt injiserer de korrekte implementasjonene. Dette gjør koden testbar fordi man enkelt kan injisere falske implementasjoner i tester.

SPØRSMÅL: "FORKLAR CLEAN ARCHITECTURE"

Clean Architecture deler koden i lag der avhengighetene alltid peker innover. I GeoScore: UI-laget (Compose + ViewModels) kjenner til domenelaget, men aldri datalaget direkte. Domenelaget (use cases + modeller) har ingen Android-avhengigheter — det er ren Kotlin. Datalaget implementerer grensesnitt definert i domenelaget. Hvis vi bytter ut Ktor med en annen HTTP-klient i morgen, trenger vi ikke å endre en eneste linje i domenelaget eller UI-laget.

Om konkrete tekniske valg

SPØRSMÅL: "HVORFOR KOTLIN COROUTINES FREMFOR THREADS?"

Coroutines er lettere enn OS-tråder. Du kan ha tusenvis av koroutiner uten å bruke mye minne. De kan suspenderes og gjenopptas uten å blokkere en tråd — viktig på Android der man ikke vil blokkere main-tråden. Med coroutines kan vi skrive asynkron kode som *ser ut* som synkron kode, noe som er mye lettere å lese og debugge enn callbacks.

SPØRSMÅL: "HVA ER ROOM OG HVORFOR BRUKER DERE DET?"

Room er Androids ORM (Object-Relational Mapping) over SQLite. Vi bruker det for lokal caching av klimadata og beregnede scores. Når brukeren ser på en adresse for andre gang, returnerer vi data fra Room umiddelbart i stedet for å gjøre nye API-kall. Historiske klimanormaler fra 1991–2020 endrer seg aldri, så cachen utløper aldri.

SPØRSMÅL: "HVA ER JETPACK COMPOSE?"

Compose er Googles moderne, deklarativ UI-rammeverk for Android. I stedet for å definere layouts i XML og deretter oppdatere Views imperativt, beskriver man i Compose UI som en funksjon av tilstanden. Når tilstanden endres, re-rendres kun de delene av UIet som er berørt. Det ligner på React for web.

Om utfordringer og læring

SPØRSMÅL: "HVA VAR VANSKELIGST MED PROSJEKTET?"

Frost API-integrasjonen var teknisk krevende — Vo og V1 returnerer helt ulike datastrukturer. Vi oppdaget at forhåndsberegnete klimanormaler ikke var tilgjengelige via API-et, og måtte aggregere 30 år med rådata client-side. Solskinndata var et spesialtilfelle der bare 36 stasjoner i Norge har data, noe vi oppdaget sent og måtte legge til eget oppslag for. Å koordinere seks utviklere med ulik erfaring var også en lærerik utfordring.

SPØRSMÅL: "HVA VILLE DU GJORT ANNERLEDES?"

Vi valgte Navigation 3 som er relativt ny og hadde lite dokumentasjon. Jeg ville undersøkt modenhetsnivået bedre. Repository-grensesnittene heter `FrostRepositoryService` i stedet for bare `FrostRepository` — navnekonvensjonen er inkonsistent. Dette og noen andre navneproblem ville jeg refaktoreert. Caching-strategien for Frost kunne også vært mer presis — vi bruker nærmeste stasjon-ID som nøkkel, men dataen er aggregert fra 10 stasjoner.

SISTE RÅD

Du trenger ikke forstå *alt* fra dag én. Fokuser på å kunne forklare GeoScore-algoritmen, MVVM/Clean Architecture, Hilt og caching-strategien flytende. Ha konkrete kodeeksempler friskt i minnet. Resten kan du si "jeg ville sett nærmere på det" — det viser ærlighet og lærevillighet.

